

AI_MarketSynthesis

FINAL 5★ SPECIFICATION

1 1. Mission

AI_MarketSynthesis is the controlled market-generation and scenario laboratory for the trading system.

Its job is **not**: “Generate fake candles.”

Its job is:

Generate realistic, statistically validated, regime-controlled alternative market trajectories that preserve the important relationships observed in real markets, then use those trajectories to test the trading system under conditions that historical data alone cannot adequately cover.

The complete relationship becomes:

```
REAL MARKET
  ↓
AI_MarketReplay
  ↓
UNDERSTAND WHAT HAPPENED
  ↓
LEARN MARKET / STRATEGY BEHAVIOR
  ↓
AI_MarketSynthesis
  ↓
GENERATE CONTROLLED ALTERNATIVE MARKETS
  ↓
RUN STRATEGY THROUGH THEM
  ↓
DISCOVER ROBUSTNESS / FAILURE
  ↓
AI_MarketReplay
  ↓
DIAGNOSE
  ↓
IMPROVE STRATEGY
```

So the two systems form a loop.

2 2. The Most Important Principle

Market Synthesis must **not** invent arbitrary random markets. It must learn from real market behavior.

Conceptually:

```
REAL DATA
  ↓
MARKET REPRESENTATION
  ↓
REGIME MODEL
  ↓
DEPENDENCY MODEL
  ↓
GENERATOR
  ↓
SYNTHETIC MARKET
  ↓
REALISM TEST
  ↓
STRATEGY TEST
```

A synthetic market is useful only if it retains the important statistical and structural properties of the real market.

Financial time series are difficult to synthesize because properties such as fat tails, volatility clustering, seasonality and cross-correlations are not trivial to reproduce. Recent diffusion approaches have shown promise, but no single approach should be assumed to solve every aspect of realistic financial generation.

3 3. Architecture

Keep the same compact architecture philosophy.

```
ai/
  marketsynthesis/
    controller.py
    config.py
    models.py
    data_engine.py
    regime_engine.py
    synthesis_engine.py
    scenario_engine.py
    validation_engine.py
    experiment_engine.py
    reporting_engine.py
    cli.py

    datasets/
    models/
    scenarios/
    artifacts/
```

```
reports/  
tests/
```

Again: not 40–50 files. The complexity should live inside the intelligent core modules.

4 4. What It Must Generate

The system should eventually generate multiple levels of market data.

Level 1 — Price

- OHLC
- returns
- volatility
- gaps

Level 2 — Market activity

- volume
- spread
- liquidity proxies

Level 3 — Multi-timeframe behavior

- M1, M5, M15, M30, H1, H4, D1

Level 4 — Structural behavior

- swings
- BOS
- CHOCH
- liquidity sweeps
- FVG
- order-block-like structures
- trend transitions
- ranges
- breakouts
- reversals

Level 5 — Strategy-observable state Where sufficient real-data relationships exist:

- VWAP, RVAM, CLV, absorption, microstructure, momentum, regime

The goal is to create a synthetic market that the existing strategy can actually consume.

5 5. Real Market Dataset

Market Synthesis should learn from a versioned dataset: `MarketDataset`

Containing:

- symbol, timestamp, OHLC, volume, spread, returns, volatility, session, timeframe

And derived information:

- ATR, VWAP, RVAM, CLV, structure, liquidity, FVG, regime, microstructure

But derived features must be calculated using the same logic and information availability rules as the live strategy.

6 6. Multi-Timeframe Consistency

This is extremely important for your system. It should **not** generate: `M1 bullish` while simultaneously producing an impossible: `H1 structure` that is inconsistent with the `M1` path.

The generator should understand: `D1 ↓ H4 ↓ H1 ↓ M30 ↓ M15 ↓ M5 ↓ M1` and maintain temporal consistency.

7 7. Market Regimes

The synthesis engine must understand regimes. Examples: strong bullish trend, weak bullish trend, strong bearish trend, weak bearish trend, range, compression, expansion, high volatility, low volatility, transition, breakout, reversal, liquidity shock.

The system must be able to request: *Generate 10,000 paths conditioned on: HIGH VOLATILITY + RANGE* or: *Generate: BEARISH TREND → LIQUIDITY SWEEP → BULLISH REVERSAL*

This is much more useful than random generation.

8 8. Scenario Engine

`scenario_engine.py` is responsible for defining controlled scenarios. Examples: `NORMAL_TREND`, `RANGE`, `FALSE_BREAKOUT`, `TRUE_BREAKOUT`, `LIQUIDITY_SWEEP`, `FVG_FILL`, `FVG_FAILURE`, `VOLATILITY_EXPANSION`, `VOLATILITY_COLLAPSE`, `TREND_REVERSAL`, `REGIME_TRANSITION`.

And combinations: HIGH_VOLATILITY + LIQUIDITY_SWEEP + FALSE_BREAKOUT

This directly targets weaknesses discovered by AI_MarketReplay.

9 9. Replay → Synthesis Feedback

This is where the two systems become extremely powerful.

Suppose Replay discovers: SMC fails frequently during: high volatility + liquidity sweep + range-to-trend transition. Market Synthesis can then deliberately generate: 10,000 high-volatility liquidity-sweep range-transition scenarios. Then run SMC against them. Now you can test the suspected weakness at scale.

10 10. Generator Architecture

Do not make diffusion the only generator. Use a hierarchy.

Layer 1 — Bootstrap / historical recombination Use real historical segments. REAL_SEGMENT A + REAL_SEGMENT B + REAL_SEGMENT C with careful continuity handling. This provides extremely realistic examples because they actually occurred.

Layer 2 — Statistical generators Use models such as: empirical return distributions, conditional distributions, volatility models, regime-transition models. These provide interpretable baselines.

Layer 3 — Conditional sequence models Learn: $P(\text{next_state} \mid \text{previous_state}, \text{regime})$. This allows controlled continuation.

Layer 4 — Deep generative model Only when sufficient data exists: Transformer, Diffusion, VAE, other sequence generator. Diffusion is particularly interesting for financial time-series generation, and recent work has explored diffusion for both financial series and order-book simulation.

But: **Diffusion should be validated against simpler generators rather than automatically declared superior.**

11 11. Conditional Diffusion

Eventually the system can support condition: regime, volatility, session, trend, liquidity state, starting window.

Then: condition → diffusion model → synthetic continuation

Example: Starting state: EURUSD, M5, trend = bearish, volatility = high, liquidity = thin, VWAP = above price, FVG = bearish, microstructure = bullish transition. Generate: 500 possible future paths.

The output is not one prediction. It is a distribution of possible futures.

12 12. Scenario Tree

The system should eventually produce:

T0

Path A → continuation

Path B → reversal

Path C → range

Path D → liquidity sweep

Path E → breakout

Path F → volatility explosion

This is much more useful for robustness analysis.

13 13. Market Synthesis Must Be Probabilistic

Do not ask: *“What will the next candle be?”* Instead: *“Given this market state, what are the plausible distributions of future paths?”*

For example: 1000 generated paths

- Bullish continuation: 37%
- Bearish continuation: 24%
- Range: 21%
- Bullish reversal: 11%
- Bearish reversal: 7%

Then evaluate strategy behavior across the entire distribution.

14 14. Synthetic Market Reality

Each generated path should contain: timestamp, price, OHLC, volume, spread, volatility, regime. and derived strategy-observable state: structure, liquidity, FVG, VWAP, RVAM, absorption, microstructure.

15 15. Realism Validation

This is perhaps the most important part of Market Synthesis. A synthetic path cannot be accepted simply because it “looks realistic.” It must pass quantitative tests.

Compare REAL vs SYNTHETIC for:

- return distribution: mean, variance, skewness, kurtosis, tail behavior
- autocorrelation
- volatility clustering
- drawdowns
- maximum moves
- gap behavior
- volume distribution
- spread distribution

And cross-feature relationships:

- return $\leftrightarrow$ volume
- return $\leftrightarrow$ volatility
- volume $\leftrightarrow$ volatility
- spread $\leftrightarrow$ volatility

16 16. Structural Realism

The system should also compare structural properties: swing frequency, BOS frequency, CHOH frequency, FVG frequency, FVG duration, liquidity sweep frequency, trend duration, range duration, reversal frequency, breakout failure rate.

This prevents the generator from producing statistically plausible but structurally unrealistic markets.

17 17. Regime Realism

Compare: REAL REGIME DISTRIBUTION vs SYNTHETIC REGIME DISTRIBUTION

For example, REAL: Trend 41%, Range 34%, Transition 15%, Expansion 10%. Synthetic should remain reasonably consistent unless deliberately conditioned otherwise.

18 18. Dependency Validation

The generator must preserve important dependencies. For example: `volatility ↑ ↔ spread often ↑` or: `volume anomaly ↔ price displacement`

The exact relationships must be learned from the actual dataset rather than hard-coded assumptions.

19 19. Market Synthesis Is Not “Fake Data”

There are three categories:

- **Historical replay:** This actually happened.
- **Recombined historical scenario:** All pieces happened, but this exact sequence did not.
- **Generative scenario:** The model believes this path is plausible given learned market behavior.

These must always be labeled separately.

20 20. Synthetic Data Provenance

Every generated path needs: `synthetic_id`, `generator_version`, `model_version`, `dataset_version`, `seed`, `conditioning_state`, `scenario_id`, `generation_timestamp`.

Example:

```
SYN-00001248
Generator: Diffusion-v2
Dataset: MarketDataset-v17
Scenario: HIGH_VOL_LIQUIDITY_SWEEP
Seed: 481923
Starting state: ...
Condition: ...
```

This makes experiments reproducible.

21 21. No Synthetic Contamination

Synthetic data must never silently contaminate OOS testing.

Correct: `REAL TRAIN` → `synthetic generation` → `development experiments` Then: **REAL OOS remains untouched.**

Otherwise you risk validating the strategy against data derived from the same information you supposedly haven't seen.

22 22. Strategy Stress Testing

Now Market Synthesis becomes extremely useful. Run strategy → 100, 1,000, 10,000, 100,000 synthetic scenarios. Then measure: failure rate, drawdown, MFE, MAE, false signals, confidence calibration, component reliability.

23 23. Component Stress Testing

You can specifically test: SMC, FVG, VWAP, RVAM, Absorption, Liquidity, Microstructure under controlled conditions.

Example: SMC

- Normal: accuracy = X
- High volatility: accuracy = Y
- Liquidity sweep: accuracy = Z
- High volatility + liquidity sweep: accuracy = W

Now you know precisely where the component is fragile.

24 24. Adversarial Market Scenarios

This is a major feature. Market Synthesis should deliberately create difficult environments: false breakout, fake BOS, liquidity sweep, rapid reversal, low-volume trap, high-spread period, volatility explosion, structure transition, FVG failure, VWAP rejection, absorption failure.

Not because these paths are guaranteed to happen, but because they are useful stress scenarios when supported by observed market patterns.

25 25. Replay + Synthesis + Code Repair

This creates the ultimate loop:

```
REAL HISTORY
  ↓
Replay
  ↓
SMC failure identified
  ↓
Root cause identified
  ↓
Candidate correction
  ↓
```

```

Market Synthesis
↓
Generate difficult SMC scenarios
↓
Run OLD SMC
↓
Run NEW SMC
↓
Compare
↓
Walk-forward
↓
Real OOS

```

This is much stronger than simply optimizing against historical trades.

26. Synthetic Counterfactuals

The system can ask: *“What if the market had continued instead of reversing?”* or: *“What if volatility had doubled?”* or: *“What if the liquidity sweep had continued another five candles?”*

The purpose is to examine strategy robustness. These should be explicitly marked as counterfactual/generated, never confused with observed history.

27. Scenario Difficulty

Each scenario should have a difficulty score: easy, normal, hard, extreme. Example: HIGH_VOL + LIQUIDITY_SWEEP + FALSE_BREAKOUT = EXTREME

This lets you build a robustness curve:

```

Strategy performance
↓
Easy      91%
Normal    82%
Hard      71%
Extreme   54%

```

The objective is not necessarily 100% performance. The objective is to understand where the strategy becomes unreliable.

28. Market Regime Generator

A particularly valuable component is a regime transition generator. Example: LOW VOL → COMPRESSION → LIQUIDITY BUILD → BREAKOUT → HIGH VOL → TREND → EXHAUSTION → REVERSAL

This allows testing the strategy during transitions, where many indicator systems become unreliable.

29 29. Multi-Asset Generalization

Eventually the generator should understand differences between assets. For example: EURUSD, GBPUSD, XAUUSD, indices, stocks.

But each asset must retain its own: volatility profile, spread behavior, session behavior, liquidity characteristics, return distribution. Never blindly pool all assets together.

30 30. Model Selection

Market Synthesis should compare generators rather than assume one is best. Example: Historical bootstrap vs Statistical model vs Regime model vs Transformer vs Diffusion.

Evaluate them using: realism, diversity, conditional accuracy, dependency preservation, tail behavior, regime preservation, strategy behavior. The winner is the generator that provides the most useful validated scenarios, not necessarily the most sophisticated model.

31 31. Diversity

A generator that creates 10,000 almost identical paths is useless. Measure: path diversity, regime diversity, trajectory diversity, tail diversity, structural diversity. You want: realistic AND diverse, not: realistic-looking copies.

32 32. Scenario Coverage

Market Synthesis should track: historically well-covered, historically under-covered, rare, extreme, unknown. This is important because the main purpose of synthesis is to expand testing where real history is sparse.

33 33. Rare-Event Laboratory

Suppose Replay discovers only 14 historical examples of: high volatility + liquidity sweep + FVG + bullish reversal. That is insufficient for strong conclusions. Market Synthesis can create a controlled scenario family around this state.

Then: 14 real examples + synthetic stress scenarios can be used for exploration. But the synthetic results must not be presented as equivalent evidence to the 14 real examples.

34 34. Confidence Separation

Reports must distinguish: REAL EVIDENCE from SYNTHETIC EVIDENCE. For example:

- Real historical evidence: HIGH confidence
- Synthetic robustness evidence: MEDIUM confidence
- Combined conclusion: PROMISING, requires real OOS confirmation

This is essential.

35 35. Market Synthesis + AI_MarketReplay

The two systems should eventually communicate.

AI_MarketReplay

failure patterns

AI_MarketSynthesis

targeted scenarios

Strategy

results

AI_MarketReplay

New diagnosis

So: Replay discovers weaknesses. Synthesis attacks those weaknesses. Replay diagnoses the results. Strategy engineering repairs the system.

36 36. Final Five-Star Ecosystem

The full architecture becomes:

REAL MARKET

FIREBASE DATA

AI_MarketReplay	Market Dataset
	AI_MarketSynthesis
	Synthetic Markets
	Strategy Stress Test
FAILURE ANALYSIS	
ROOT-CAUSE ENGINE	
SOURCE-CODE TRACE	
CORRECTION HYPOTHESIS	
CANDIDATE PATCH	
REPLAY + SYNTHESIS	
A/B + ABLATION	
WALK-FORWARD	
REAL OOS	
ACCEPT / REJECT	

37 37. The Ultimate Objective

The ultimate system is therefore not: Trading AI. It is: **MARKET INTELLIGENCE LABORATORY** with three major capabilities:

AI_MarketReplay Understand reality. What happened? Why? Where did we fail? Which com-

ponent failed? Which code caused it?

AI_MarketSynthesis Explore plausible reality. What other market paths are plausible? What happens under rare conditions? Where is the strategy fragile?

Strategy Repair Loop Improve reality performance. What should we change? Does the change fix the original problem? Does it create new problems? Does it survive synthetic stress? Does it survive real OOS?

38 38. Final Principle for Market Synthesis

The most important rule is: **Synthetic markets are a laboratory, not evidence that the future will behave that way.**

Real historical data remains the primary evidence. Synthetic generation is used to: expand scenarios, stress the system, test robustness, explore rare regimes, find hidden weaknesses, test candidate repairs.

And because current research still identifies simulation-to-reality gaps and evaluation difficulties, the synthetic generator itself must undergo rigorous realism validation rather than being trusted simply because it uses a sophisticated generative model.

So the final relationship is: REALITY → REPLAY → UNDERSTANDING → SYNTHESIS → STRESS → DIAGNOSIS → REPAIR → VALIDATION → REAL OOS → NEW REALITY → REPLAY AGAIN

That is the 5★ Market Synthesis target I would build alongside the final system.